ADAPTOFLUX: A TYPED GRAPH SUBSTRATE FOR STATEFUL PROGRAM CONSTRUCTION AND REUSE

Anonymous authors
Paper under double-blind review

ABSTRACT

AdaptoFlux is a typed graph substrate for constructing and reusing stateful symbolic programs. It separates editable program graphs from an extensible method pool: declared types guide candidate connections, constructed subgraphs can be registered for subsequent composition, and zero-output action nodes participate in execution without a path to the numerical output. External state connects successive graph executions. Search procedures propose graph structures and method choices; optimized numerical parameters can also be embedded in the resulting programs. Experiments examine semantic type priors, module reuse, execution scale, and integration with an existing graph system. On the studied stateful synthesis task, fine-grained, role-merged, and all-scalar declarations achieve success in 29/30, 15/30, and 1/30 runs. Execution measurements cover up to 1,024 operation nodes. A comparison with PyTensor completes the same discover–register–reuse workflow and reproduces four frozen action controllers over 21,200 steps, with identical actions and zero differences in features, scores, and memory. Controlled search separately identifies benefits of a shared module-reuse policy. An optional resource-ordering mechanism supports consistent shared-state execution. These results characterize concrete interfaces for program modification, reusable computational structure, and stateful interaction.

1 INTRODUCTION

Symbolic program synthesis searches over discrete executable structures. Its representation determines which operations can be composed, where intermediate results can be reused, and how a program interacts with mutable state. These choices matter for control tasks: a controller must combine numerical computation with observations, action selection, and environment updates, often over many successive steps. AdaptoFlux makes these interfaces explicit through a typed Method Pool and Generalized Data Flow Graphs (GDFGs).

Consider an action controller that reads an observation o_t and an internal state h_t, computes shared features z_t, scores each available action, and executes the action selected by an exclusive router. One feature can contribute to several action scores without being recomputed. The state-update branch commits h_{t+1} after reading h_t, and the same stored graph runs at the next step. This structure combines typed dataflow, learned numerical parameters, and stateful effects in a single program (Figure 1).

The representation exposes two objects to learning procedures: a program graph and the method pool from which its nodes are chosen. A graph edit changes a particular program; registering an extracted subgraph expands the vocabulary available to later programs. Declared types constrain candidate connections, and method metadata guides candidate selection. Numerical outputs and environment effects are both observable program behavior: an action branch can matter even when it contributes no returned value. The optional execution contract separately orders conflicting shared-state accesses.

The paper makes three contributions. First, it specifies how typed method declarations, editable graphs, and subgraph registration form a common program-construction interface. Second, it defines execution that includes zero-output actions and repeated interaction with external state; resource-conflict ordering provides an optional consistency extension. Third, controlled studies distinguish the roles of type priors and module reuse, measure execution scale, and compare workflow and action

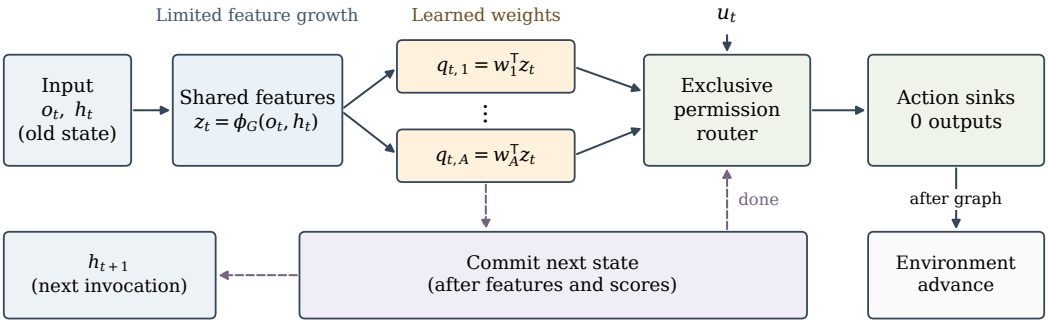


Figure 1: A concrete GDFG controller. Shared features feed per-action scores; an exclusive router activates one action. The state branch reads the old state and commits the next state. Dashed purple paths show commit/control relations. The template supplies these interfaces, CMA optimizes score coefficients, and scheduled proposals can extend the feature graph.

interfaces with an existing graph system. LayerGrow, GraphEvo, and the CMA-based controller optimizer illustrate different procedures operating on these executable structures.

The implementation is available through the anonymous review repository at `https://anonymous.4open.science/r/AdaptoFlux-CF54`. Protocols and the provenance of the supplementary measurements are specified in Appendices A–I.

2 Positioning and Related Work

Typed program composition, graph reuse, and effect ordering have substantial prior foundations. Strongly typed genetic programming constrains compositions through types Montana (1995); Cartesian Genetic Programming provides graph-based reuse Miller & Turner (2015); and MT-CGP explicitly supports mixed data types Harding et al. (2012). LLVM SelectionDAG combines typed dataflow with chain dependencies for side-effect ordering LLVM Project (n.d.). These precedents motivate our distinction between data edges and control dependencies.

AdaptoFlux organizes these mechanisms around a method pool and editable program graph. DreamCoder learns reusable abstractions in a functional language Ellis et al. (2021); Popper uses failure-driven constraint learning Cropper & Morel (2021); symbolic-policy and language-model-guided discovery emphasize search quality Landajuela et al. (2021); Romera-Paredes et al. (2024). PyTensor PyTensor Developers (2026) provides typed computational graphs, native subgraph encapsulation, and Python callbacks. Our comparison identifies concrete interface choices: ATF retains input-ready zero-output actions in graph execution, while the tested PyTensor paths retain effects through requested outputs or shared-variable updates. Both support the tested reuse and action workflows without core changes (Section 4.8). Future work may use language models as candidate generators (Appendix S). Appendices J and N cover further comparisons, including gplearn and PySR Cranmer (2023).

3 System Architecture

3.1 Method Pool

A method is registered as $m = \langle \mathrm{impl}, \sigma_{\mathrm{in}}, \sigma_{\mathrm{out}}, \mathrm{M} \rangle$, comprising its implementation, positional input and output type signatures, and metadata. Types are declared atoms, such as `scalar`, `pos_type`, or `target_type`. Each input position lists the atoms it accepts; each output port declares the atom it emits. Arity and positional routing determine which outputs can supply which inputs.

Metadata has two roles. Search annotations, including groups and sampling weights, select a view $\mathrm{Q}_{\mathrm{view}} \subseteq \mathrm{Q}$ of the pool. Execution annotations declare the resources a method reads and writes,

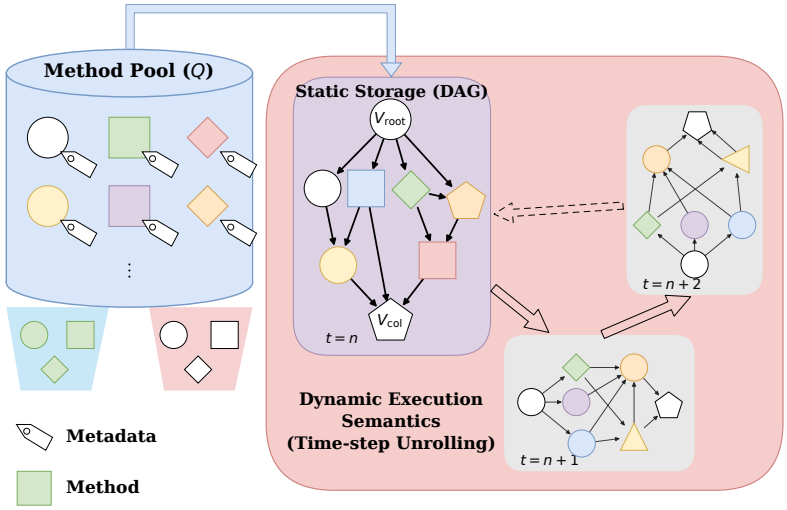


Figure 2: AdaptoFlux architecture. The Method Pool defines callable interfaces, the stored GDFG carries data dependencies, and the runtime connects successive steps through state. Effect-ordering plans add control dependencies to the per-step schedule.

denoted R_m and W_m, or identify a method as pure. Undeclared effects are treated conservatively as unknown. The same resource name must describe accesses to the same external state. A registry entry can also contain a reusable subgraph; its effect contract aggregates those of its constituents. Appendix Q.4 separates these execution contracts from search preferences.

3.2 GDFGs and Stateful Execution

A data graph $G = \langle V, E_d \rangle$ is a DAG with input, operation, and output-aggregation interfaces. Every operation node is bound to a registered method. An edge connects a particular output port to a particular input position, and is admissible when its emitted type belongs to that position's accepted type set. The data-graph hypothesis class is

$$\Omega_{\mathrm{Q},\Sigma} = \{G : \ G \text{ is a DAG}, \ m_v \in \mathrm{Q}, \ \text{and every data edge respects its port types}\}. \tag{1}$$

Declared type compatibility is checked during graph construction. The declarations are callable interface contracts: method implementations must also respect their declared inputs and outputs.

Numerical outputs and action execution. The runtime executes processing nodes with available inputs, including zero-output action nodes that have no path to the numerical output. The `collapse` interface collects returned values separately; a valid V0 invocation still requires a connected aggregation interface. An action therefore needs no extra output solely to retain its execution. This rule determines participation in execution, while the ordering contract below determines the order of conflicting state accesses.

State transitions require a selected order. For fixed input x_t, a node computes data outputs and may update the current external state. Let $\pi = (v_1, \ldots, v_m)$ be a selected topological order. Writing F_v for the state component of node execution, one step is

$$s^{(0)} = s_t, \qquad s^{(k)} = F_{v_k}(\mathbf{in}_{v_k}, s^{(k-1)}, x_t), \qquad s_{t+1} = s^{(m)}. \tag{2}$$

The choice of π matters. Starting from $s = 1$, independent nodes that apply $s \leftarrow s + 1$ and $s \leftarrow 2s$ produce either 4 or 3. Both orders may satisfy the same data DAG. We therefore define the stateful executable program by $P = (G, \pi, E_c)$, where E_c contains control dependencies that preserve the selected order of conflicting accesses.

Optional conflict-aware scheduling. Two nodes conflict when

$$W_u \cap (R_v \cup W_v) \neq \varnothing \quad \text{or} \quad W_v \cap R_u \neq \varnothing. \tag{3}$$

For each conflicting pair with $u \prec_\pi v$, the augmented graph $G^+ = (V, E_d \cup E_c)$ contains a dependency path from u to v. The implementation constructs these paths using resource read/write frontiers; control dependencies carry completion information, while function arguments come only from E_d. Shared reads and accesses to disjoint resources remain available for parallel execution. Unknown methods are conservatively ordered with other effectful methods.

With complete effect declarations, deterministic operations, the same inputs and initial state, and successful execution, legal schedules of G^+ produce the same outputs and final state. Adjacent independent operations can be exchanged without changing their inputs or resource observations; conflicting operations retain their order. Appendix A gives the execution conditions and validation. If operation-node updates admit increments I_v applied by $\oplus$, and those increments can be evaluated from the same pre-step state and commute on disjoint resources, Equation 2 reduces to

$$s_{t+1} = s_t \oplus \bigoplus_{v \in V_{\text{proc}}} I_v(\mathbf{in}_v, s_t, x_t). \tag{4}$$

Training and persistence. Before a candidate is evaluated, the runtime freezes its initial order. Stable node identities retain that order through accepted substitutions, graph growth, copying, and save/load operations. An explicit order change defines a new candidate for evaluation. For effectful batches, the runtime finishes a complete sample before starting the next. The effect-ordering feature is enabled by default and can be disabled; the selected mode is stored with the model. Disabled mode follows data dependencies and node-wise batching, with no additional state-ordering guarantee.

Across time steps, external state or explicit state inputs connect successive executions of the same stored graph. A finite unrolling uses forward-in-time connections and remains acyclic (Appendix Q.6). The hand-written Snake example uses mutually exclusive action permits; the controller in Figure 1 additionally exposes a read-old/commit-new internal-state boundary.

3.3 Navigation and Reuse

LayerGrow samples type-compatible operations to extend a graph, evaluates the candidate, and accepts or rolls back the extension. **GraphEvo** refines a fixed graph skeleton by replacing its bound methods, and can extract subgraphs into reusable methods. With effect ordering enabled, extracted methods retain their internal order and resource declarations; fragments interleaved with external effectful nodes are rejected (Appendix A). **CombinedTrainer** composes growth and refinement with pool filtering. Their algorithmic descriptions and original experiments are retained in Appendices K.2 and L. The action-controller study uses CMA to optimize numerical score coefficients and scheduled proposals to grow arithmetic features, illustrating a second optimization interface on the same form of executable structure.

3.4 Execution Cost and Parallelism

For V operations, E_d data edges, E_c control edges, batch size N, and average operation cost $\bar{C}_{\text{op}}$, inference separates plan preparation, scheduling, routing, and computation:

$$T_{\text{infer}} = T_{\text{prepare}} + T_{\text{sched}}(N, G^+) + O\big(N(|E_d| C_{\text{route}} + V \bar{C}_{\text{op}})\big). \tag{5}$$

The revised runtime validates the graph and reconstructs its plan on each call. Pure graphs use one batched traversal, whereas effect-ordered stateful batches use N complete traversals. The threaded implementation sorts ready nodes and creates a worker pool for each traversal; these costs belong to T_{sched}. Resource declarations and the selected ordering determine preparation and control bookkeeping. The historical backend's measurements and complexity breakdown are given in Appendix K.

Parallel benefit depends on the ready graph width, critical path, operation cost, and backend. The original simulated non-blocking workload achieved a mean $10.25\times$ speedup with 16 workers. The supplementary Python scalar CPU workload instead runs faster serially (Section 4.6). Reporting both regimes distinguishes available graph concurrency from the cost of realizing it.

3.5 Search-Space Structure

Type routing removes inadmissible port compositions before evaluation. Search annotations then select among admissible candidates; effect contracts determine their execution. Increasing the pool can introduce useful operations and simultaneously increase the choices explored under a fixed budget. LayerGrow's cost accumulates across repeated graph extensions, while GraphEvo refinement holds the skeleton fixed. The experiments below measure these effects separately from execution scalability.

4 Experimental Results

We report three evidence groups: the original navigation experiments, supplementary controlled measurements, and applications developed with additional optimizers. The four supplementary studies of type priors, state encoding, execution scaling, and controller replay used the original frozen ATF backend. The revised effect-aware runtime has a separate regression suite and frozen-model replay audit. Appendix A records this version separation.

4.1 Structural Search with LayerGrow

On Titanic Cukierski (2012), LayerGrow uses nine scalar operations and a 2,500-attempt budget per run. Across ten runs per maximum-layer setting, mean best accuracy reaches 0.763 ± 0.041 at a configured maximum of 20 layers. Higher requested maxima produce little further improvement (means 0.769–0.783 for settings 25–50) and more budget exhaustion. At requested maxima 45 and 50, all runs exhaust the attempt budget; the mean reached depths are 19.6 and 18.7. Thus the observed failure concerns this growth procedure's progress under its budget. Figure 4 and Appendix L.1 give the full depth and cost results.

4.2 GraphEvo and Reusable Subgraphs

The task chain is $f_1(x) = 2(x + 1)$, $f_2(x) = f_1(x) + 3$, and $f_3(x) = 0.5 f_2(x)$. With two-layer initialization for Tasks 1/2, extracted methods yield Task 3 success in 13/100 runs, compared with 0/100 without extraction and 12/100 with a supplied prior. Task 3 always permits at most three top-level calls. Exact enumeration shows that the fixed primitive pools require four calls to express $x + 2.5$; composite methods change reachability under that bound. These results measure fixed-depth abstraction (Appendix L.2).

A separate capacity-matched study gives ATF and an independently implemented reference DAG the same searcher, eight expanded primitive operations, and 256 evaluations per stage. Across 30 seeds per condition, growing versus fixed libraries yield final affine success in 10/30 versus 2/30 runs in both backends, with identical paired search traces. The benefit belongs to the shared module-admission policy; complete results and costs are in Appendix I.

4.3 Semantic Type Priors in Stateful Synthesis

The supplementary Minimal Stateful Attractor experiment holds nine operator implementations fixed and varies only their declared types. The conditions distinguish position and target roles, merge them into one quantity type, or represent all values as scalars. Each condition uses the same 30 search seeds. Each evaluation exhaustively tests the 20 unequal start/target pairs in $\{-2, -1, 0, 1, 2\}$, with at most four graph steps per episode. Strict run success requires solving all 20 pairs.

Table 1 shows a clear contribution from semantic role information: merging position and target types reduces success from 29 to 15 runs, and flattening all types reduces it to one. Initialization already solves 20, 4, and 0 runs, respectively; refinement supplies the remaining successes. Type declarations improve both the initial candidate distribution and the set of permitted replacements. The realized evaluation budgets differ because stopping conditions and available replacements differ. Appendix B reports these budgets, seed-level outcomes, and final-model audits. The original 10-seed experiment, which used a different evaluation protocol and obtained 8/10 versus 0/10, is retained separately in Appendix L.4.

Table 1: Semantic type priors improve stateful synthesis. All conditions use the same nine operator bodies and nominal search caps. Evaluation calls report the realized cost; intervals are Wilson 95% intervals across 30 runs.

Type declarations	Successful runs	95% interval	Mean calls
Position / target	29/30 (96.7%)	83.3–99.4%	59.47
Merged quantity	15/30 (50.0%)	33.2–66.8%	81.67
All scalar	1/30 (3.3%)	0.6–16.7%	738.67

Table 2: Nested atomic-pool expansion. Numerical pass rate averages ten task families and four seeds; maze completion averages two seeds. Both values are percentages.

Operators	Numerical input pass rate	Maze completion
6	14.01	11.43
9	11.45	58.01
15	7.72	37.89
20	11.35	6.93

4.4 Execution Consistency under Alternative Schedules

The optional effect-ordering executor preserves a candidate's selected access order through acceptance, copying, and reloads. Replaying 90 frozen models on all 20 initial conditions in serial and three-worker modes matches every trajectory and saved metric: 1,800 model–initial-state pairs and 4,934 paired steps. A six-operation mixed-resource graph agrees under every legal schedule, while a barrier test confirms concurrent execution of independent branches. Targeted tests cover conflicting writes, shared reads, unknown effects, graph edits, and persistence; the recorded regression suite passes 85 tests and 46 subtests. Appendix A gives the execution conditions and complete validation separately from V0 measurements.

4.5 Implicit and Explicit State Encodings

For 1,007 candidate/final-model records, implicit accesses are translated into immutable state inputs and outputs while preserving access order. All node values, states, trajectories, and final metrics match across 20,140 paired initial conditions and 70,670 steps. Explicit encoding adds no operation nodes and a mean of 2.189 state edges over the 90 final graphs; its paired execution-time ratio has median 1.137, including state construction and dispatch. Altering the unrelated global state during explicit execution leaves its results unchanged. This measures execution correspondence for already selected programs. Appendix C gives the translation and timing protocol.

4.6 Scaling Graphs, Pools, and Execution Workloads

Single-graph execution. At fixed depth eight and batch size 32, we vary width to obtain 32–1,024 operation nodes and 68–2,176 edges, with five graph seeds at each size. Serial time increases from 2.127 to 70.586 ms across this 32-fold node increase. At the largest size, a separate declared-type compatibility pass takes 1.448 ms and topological preparation takes 2.310 ms. Four-worker time is 82.379 ms: threading is slower for these Python scalar CPU operations. All 30 graphs agree with an independent numerical reference (maximum absolute error 2.22×10^{-16}). Figure 3 shows the measured curves; 1,024 nodes is the largest tested point, and the overhead-dominance threshold has not been reached in this sweep.

Atomic operator-pool expansion. A follow-on search implementation compares nine pools containing 6–20 operators across 12 numerical tasks and four action tasks. Pool expansion has a non-monotonic effect under fixed candidate and interaction budgets (Table 2). The six-operation arithmetic core gives the strongest aggregate numerical pass rate along the nested expansion; adding increment, decrement, and negation gives the strongest maze result. The different responses show that useful operator coverage and the distribution of candidates jointly shape search quality.

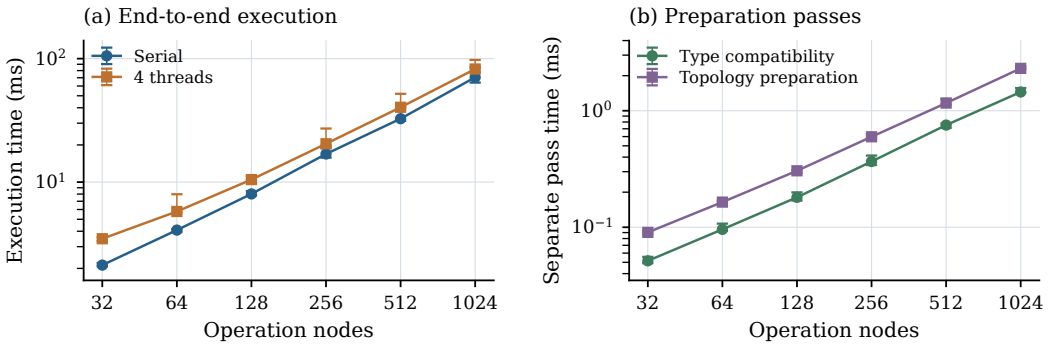


Figure 3: Fixed-depth scaling on the original backend. Medians summarize five graph seeds per size; error bars show their minimum–maximum range. Each metric uses two warm-ups and seven repetitions. The declared-type pass and topology preparation are measured separately; inference includes its own preparation. Full measurements are in Appendix D.

Table 3: Library capacity in the 40-stage application. Costs count environment interactions and average ten paired seeds; lower is better. Return cost measures recovery of previously encountered rule families.

Configuration	Capacity	Mean attainment cost	Return cost
Direct reuse	32	1,792.32	1,749.76
Direct reuse	128	1,691.52	1,512.96
Full combination	32	1,855.04	1,802.88
Full combination	128	1,683.20	1,472.00

Numerical conditions use 131,072 training input presentations per task and seed, 256 candidate opportunities, and a 32-node graph cap. Action conditions use 65,536 training episodes, 128 candidate opportunities, and a 24-node cap. Numerical success in the table is the fraction of test inputs with relative error at most 1%. The extensions change operator composition and arity along with pool size. Appendix F gives the full nine-pool comparison and the path-feedback procedure. This experiment identifies a practical design choice: pool views should expose operations relevant to the task while controlling the alternatives presented to a finite-budget optimizer.

Sustained execution and reusable libraries. A continual-learning application runs 80 sequences of 40 stages across ten training seeds. Each observation has 64 fields, one of 32 identities, and six available actions. The sequence presents 20 rule families and then returns to them in reverse order. The completed runs record 9,830,400 valid training actions and 229,376,000 evaluation input rows: 239,206,400 input rows in total undergo native-ATF risk-output checks. These counts measure application workload; graph calls can process whole arrays.

Attainment cost counts interactions up to the earliest checkpoint after which balanced risk accuracy of at least 90% and optimal-action rate of at least 80% in the carrying condition are maintained. A stage that never attains this criterion receives a cost of 3,200. Increasing capacity from 32 to 128 lowers mean attainment cost by 5.62% for direct reuse and 9.26% for the full configuration (Table 3). The full configuration's return cost decreases by 18.35%, while its first-exposure cost changes little. Additional retained structures therefore help most when earlier tasks are revisited. At capacity 32, the full configuration does not meet its predeclared advantage criterion over direct reuse. A separate shorter capacity sweep shows no stable gain; its capacity-32 and capacity-64 runs produce identical trajectories because occupancy never exceeds 21 entries. The results distinguish retention capacity from the search and selection mechanism used with it. Appendices G and H provide all eight conditions and the short-sequence comparison.

Table 4: Held-out action-task completion (%). Each trained model is evaluated on 2,048 episodes. Mean and range summarize five training seeds; individual results are in Appendix E.

Task	Completion criterion	Step cap	Mean	Seed range
Catch	Catch all 16 balls	160	92.10	61.28–100.00
Maze	Reach $(7, 7)$	1,024	92.74	75.98–98.88
Snake	Collect 8 food items	512	43.55	0.05–93.12
Bird	Pass all 32 pipes	1,024	100.00	100.00–100.00

4.7 A Learned Action-Scoring Controller

The structure in Figure 1 computes

$$z_t = \phi_G(o_t, h_t), \qquad q_{t,a} = \sum_j w_{a,j} z_{t,j}, \qquad a_t = \mathrm{route}(q_t, u_t). \tag{6}$$

The router uses a supplied draw u_t when its selection rule is stochastic. The template supplies shared features, per-action score branches, exclusive permits, zero-output action sinks, and state-update interfaces. CMA learns the score coefficients; scheduled structure proposals can add arithmetic features or change state-emission sources. Table 4 reports held-out completion across five training seeds, with a separate model for each task.

Bird succeeds consistently, while Snake exposes substantial training-seed variation. In the validation-selected representative system, Catch, Maze, and Snake have no subsequent feature-growth event; Bird has four. Its 44 shared features supply 88 coefficients. For example, $z_{16} = \texttt{gapBottom} + h_1^{\mathrm{old}}$ enters the flap and coast scores with weights approximately $+3.001$ and -6.538, directly illustrating how one state-dependent calculation serves competing action branches. Feature proposals occur at fixed training-budget milestones, so the contribution of optimized coefficients and the occurrence of structural growth can be inspected separately in the saved controller.

A frozen bird model selected by validation was adapted from 314 nodes/411 edges to 316 nodes/565 edges by making its input, state-commit, routing, and observation interfaces explicit. Running this adapted graph in the original ATF executor matches the source controller and an independent interpreter for 20,480 steps over 20 fixed test episodes, with zero difference in features, scores, state, and actions. The replay connects the learned controller to the original execution interface. Appendix E gives the five-seed results and adapter specification.

The adapter exposes a complete control step: feature and score nodes read the old state; a single commit node writes the next state; a completion dependency releases the router; the selected action sink records the action; and the environment advances after graph execution. Thus shared computation, state update, and action emission each have an explicit place in the program. The example shows how the representation carries a learned numerical controller while preserving its discrete execution structure.

4.8 Comparison with a Typed Computational-Graph System

We implement the same discover–register–reuse workflow in frozen ATF V0 and PyTensor 3.3.8. Each backend evaluates its own candidates and wraps successful programs using `EvolvedMethod` or `OpFromGraph`. Twelve method-order cases (11 distinct permutations) produce 36 paired stages and 2,638 candidate evaluations with matching traces. Both systems reuse nested modules and reconstruct programs from common JSON declarations without core changes. Search, admission, and declaration persistence belong to the shared outer procedure.

Four frozen controllers are also expressed through native numerical nodes in both backends. Actions, environment states, features, scores, and memory agree over 21,200 steps; the maximum numerical difference is zero (Table 5). Bird additionally matches the existing 20,480-step archive. This is execution migration, with no controller retraining.

The action interface differs in how effects remain executable. ATF retains input-ready zero-output sinks; PyTensor retains the tested callbacks through requested outputs or shared-variable updates. Both mutate the same small environment correctly. PyTensor's `wrap_py` decorator also passes

Table 5: ATF V0 and PyTensor interface checks. Controller episodes are fixed evaluation-stream prefixes; counts are coverage, not independent training trials.

Check	Paired coverage	Result
Numerical reuse workflow	36 stages	Identical traces
Catch / Maze / Snake	720 steps	Identical behavior
Bird	20,480 steps	Identical behavior
Routing and state boundaries	49 cases	All matched

this test without handwritten operator classes, with either no optimization or `FAST_RUN`; updates allow empty user-visible outputs. Unanchored callback branches are not executed. This distinguishes execution conventions, while ordering conflicting actions remains a separate requirement. Appendix I records protocols, native interface boundaries, and the separate reference-DAG search control.

LIMITATIONS

The stateful synthesis task isolates a supplied semantic type prior in a small environment. A remaining comparison is independent search with an explicit-state GP/CGP system under matched realized budgets. The larger applications use additional optimizers and task templates, and their performance depends on these choices. Effect ordering relies on accurate resource declarations; arbitrary Python side effects and runtime exceptions are outside static type compatibility. New-runtime preparation overhead requires its own performance measurements. The graph sweep has not located the practical upper bound or independently varied edge density. Python CPU threading is slower in the measured scalar workload, while the restricted static symbolic-regression benchmark favors specialized solvers on several formulas. These findings identify concrete next steps in search, backend engineering, and evaluation.

5 CONCLUSION

AdaptoFlux organizes editable program graphs, an extensible method pool, and stateful execution through common interfaces. Declared types participate in construction, extracted subgraphs become reusable methods, and zero-output actions participate directly in graph execution. The experiments distinguish supplied type priors, module-reuse policies, execution costs, and integration choices. The PyTensor comparison demonstrates shared workflow capabilities and a concrete difference in retaining effectful branches. Optional resource ordering supplies a separate consistency contract for conflicting state accesses.

Future work will investigate continually learning agents that combine different relation-discovery procedures through this program and method-library interface. The goal is to retain learned subgraphs with applicability conditions and supporting evidence, and use unexpectedly beneficial outcomes and contradictory feedback to guide exploration and local revision. This aims to support knowledge accumulation and adaptation across changing tasks within bounded memory and computation budgets.

AI USE STATEMENT

AI assistants supported the revision through implementation assistance, supplementary experiment scripting, result checking, literature lookup, English drafting and editing, and figure and layout preparation. Numerical results are derived from saved execution records and analysis scripts. The software checks include matched-state replay, independent numerical references, and regression tests. The authors are responsible for the paper's claims and the verification of AI-assisted material.

ETHICS STATEMENT

This work follows the ICLR Code of Ethics. It uses public benchmark data and synthetic tasks, involves no human-subject intervention, and reports no personally identifiable experimental data. Automatically generated programs can invoke external actions; applications should validate registered methods and isolate execution environments. Appendix U discusses interpretability, misuse, and bias inherited from supplied methods.

REPRODUCIBILITY STATEMENT

The appendices specify method pools, seeds, evaluation criteria, runtime versions, and measurement protocols. The supplementary type study retains final graphs and complete evaluation traces; the state audit retains paired trajectories; the scaling study retains per-seed repetitions and independent reference checks; and the controller case retains its frozen model and stepwise replay. The original implementation and experiment materials are linked through the anonymous repository. Additional evidence is identified by its protocol and artifact names in Appendices A–I; its execution records are kept separate from historical experiments. Data-graph type checks, effect-ordering assumptions, and finite-unrolling arguments are specified in Sections 3.2–3.4 and Appendix Q.6.

REFERENCES

Miles Cranmer. Interpretable machine learning for science with PySR and SymbolicRegression.jl. *arXiv preprint arXiv:2305.01582*, 2023.

Andrew Cropper and Rolf Morel. Learning programs by learning from failures. *Machine Learning*, 110(4):801–856, 2021.

Will Cukierski. Titanic - machine learning from disaster. `https://kaggle.com/competitions/titanic`, 2012. Kaggle.

Kevin Ellis, Catherine Wong, Maxwell Nye, Mathias Sablé-Meyer, Lucas Morales, Luke Hewitt, Luc Cary, Armando Solar-Lezama, and Joshua B Tenenbaum. Dreamcoder: Bootstrapping inductive program synthesis with wake-sleep library learning. In *Proceedings of the 42nd acm sigplan international conference on programming language design and implementation*, pp. 835–850, 2021. doi: 10.1145/3453483.3454080. URL `https://doi.org/10.1145/3453483.3454080`.

Simon Harding, Vincent Graziano, Juergen Leitner, and Juergen Schmidhuber. MT-CGP: Mixed type cartesian genetic programming. In *Proceedings of the 14th Annual Conference on Genetic and Evolutionary Computation*, pp. 751–758. ACM, 2012. doi: 10.1145/2330163.2330268.

Mikel Landajuela, Brenden K Petersen, Sookyung Kim, Claudio P Santiago, Ruben Glatt, Nathan Mundhenk, Jacob F Pettit, and Daniel Faissol. Discovering symbolic policies with deep reinforcement learning. In *International Conference on Machine Learning*, pp. 5979–5989. PMLR, 2021.

LLVM Project. The LLVM target-independent code generator: Introduction to SelectionDAGs. `https://llvm.org/docs/CodeGenerator.html#introduction-to-selectiondags`, n.d. Accessed November 2026.

Julian Miller and Andrew Turner. Cartesian genetic programming. In *Proceedings of the Companion Publication of the 2015 Annual Conference on Genetic and Evolutionary Computation*, pp. 179–198, 2015. doi: 10.1145/2739482.2756571. URL `https://dl.acm.org/doi/10.1145/2739482.2756571`.

David J Montana. Strongly typed genetic programming. *Evolutionary computation*, 3(2):199–230, 1995.

PyTensor Developers. PyTensor 3.3.8. `https://github.com/pymc-devs/pytensor/releases/tag/rel-3.3.8`, 2026. Version used in the computational-graph comparison.

Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M Pawan Kumar, Emilien Dupont, Francisco JR Ruiz, Jordan S Ellenberg, Pengming Wang, Omar Fawzi, et al. Mathematical discoveries from program search with large language models. *Nature*, 625(7995):468–475, 2024.

Silviu-Marian Udrescu and Max Tegmark. Ai feynman: A physics-inspired method for symbolic regression. *Science Advances*, 6(16):eaay2631, 2020. doi: 10.1126/sciadv.aay2631.

A State-Access Ordering and Execution Consistency

The revised executor makes the order of conflicting state accesses part of the stored program. This supplies the missing ordering information when two nodes have no data-dependency path but access the same external state. The original data edges continue to carry method arguments; additional control dependencies only determine when a node may execute.

Execution versions and evidence. We distinguish the original executor, denoted V0, from the revision with optional effect ordering, denoted V1. The three-level type experiment, matched state encoding experiment, CPU scaling measurement, and frozen-controller replay below were completed with V0 and frozen protocols. Their measurements remain associated with that implementation. V1 is evaluated separately by execution-consistency tests and frozen-model regression. The subsequent pool and continual-learning studies use their stated search procedures around the ATF representation and execution interfaces. This separation identifies which implementation produced each result while retaining the original experimental records.

Program order and conflict relation. For each method instance v, let R_v and W_v be its declared read and write resource sets. Two instances conflict when

$$W_u \cap (R_v \cup W_v) \neq \varnothing \quad \text{or} \quad W_v \cap (R_u \cup W_u) \neq \varnothing. \tag{7}$$

Before the first evaluation of a candidate, a topological order τ of its data graph is selected and stored using stable node identities. For each conflicting pair, control dependencies preserve its order in τ. The implementation maintains the latest writer and outstanding readers for each resource; transitive dependencies avoid storing every conflicting pair. Unknown effects form conservative barriers among effectful methods, and a wildcard resource denotes access to all resources. Read-only accesses and accesses to disjoint resources retain their available parallelism.

All data and control dependencies point forward in τ, so their union is acyclic. Assuming deterministic methods, complete resource declarations, and identical initial state and inputs, successful executions respecting this union produce the same outputs and final state. To see this, any two legal linear extensions are connected by adjacent swaps of incomparable nodes. Such nodes have neither a data dependency nor a conflicting state access, so each swap preserves their outputs and state updates. Repeating these swaps gives the claim. Resource declarations therefore concern actual shared objects, including random-number streams and external callbacks; type compatibility alone does not establish this property.

Counterexample and positive control. Starting at $s = 1$, two independent data branches that perform $s \leftarrow s + 1$ and $s \leftarrow 2s$ yield 4 or 3 under opposite topological orders. Declaring both branches as writers of the same resource introduces the chosen control order and preserves the corresponding result in serial and threaded execution. The test suite also uses synchronization barriers to verify that independent branches actually overlap in time. It exercises read/write conflicts, wildcard resources, zero-output actions, graph edits, storage reordering, and failure propagation.

Training and persistence. Candidates are prepared before evaluation. An accepted candidate retains the evaluated graph and its semantic order; method replacement preserves stable node identities. Graph growth, replacement, and subprogram extraction validate the resulting dependencies. Explicitly changing the order creates a new candidate to evaluate. Saved plans contain the order, ports, method signatures, and effect declarations; loading checks these contracts before replacing the live model. The plan stores execution semantics rather than a copy of the external environment.

Why stateful extraction needs an execution contract. Consider a stored order $A \prec B \prec C$, where A increments s, B observes s, and C doubles s. Starting from $s = 1$, B observes 2 and the final state is 4. Extracting A and C into one uninterrupted composite call leaves B observing either 1 or 4, although the numerical ports can still be compatible. With effect ordering enabled, the extractor therefore rejects effectful fragments interleaved with external effectful operations. Supported composites retain their internal order and aggregate resource declarations, which are checked again on reuse and reload. This makes preservation of surrounding observations a condition of subprogram reuse. The extraction and persistence checks are exercised in `Test/test_effect_training.py`.

Table 6: Type resolution under the frozen V0 search protocol. Wilson intervals describe the run-level success proportion. Evaluation counts are means per run.

Type condition	Successes	95% interval	Evaluations	Distinct graphs
Fine	29/30	83.3–99.4%	59.47	6.87
Coarse	15/30	33.2–66.8%	81.67	5.60
Flat	1/30	0.6–16.7%	738.67	22.77

Optional execution mode. V1 enables effect ordering by default and records the choice with the model. With ordering enabled, a stateful batch completes one sample's graph before the next sample begins. With ordering disabled, scheduling uses data edges alone and retains node-major batch processing. Both modes validate graph ports and preserve the chosen setting through training, cloning, and persistence. Switching modes changes the program's execution contract and calls for a new evaluation. The ordering guarantee applies to successful executions; external effects already performed before a method failure are not rolled back.

Frozen-model regression. All 90 final models from the type experiment were replayed in V1 on their 20 initial conditions. Across 1,800 model–initial-state pairs and 4,934 paired environment steps, serial and three-worker execution matched in trajectories, accuracy, and loss, including the saved V0 outcomes. The associated implementation suite passes 85 tests and 46 subtests. These results validate the integrated execution path without repeating model training.

B Effect of Type Resolution on Stateful Search

Separating the roles of position and target produces the strongest search result in the finite attractor task: 29 of 30 runs solve every initial condition. Merging those roles lowers success to 15 of 30, and using scalar types throughout lowers it to 1 of 30. The experiment directly exposes the contribution of the supplied type prior.

Protocol. The three conditions share the nine original method implementations, arities, groups, weights, and registration order. Fine types distinguish `pos_type` and `target_type`. Coarse types merge both into `quantity`, which remains distinct from `scalar`. Flat types use `scalar` throughout. Every condition retains the goal-aware movement operator. Formal seeds are 110600–110629; pilot seed 110500 is excluded. Python and NumPy random generators are reset to the run seed for each condition.

GraphEvo uses three fixed-mode initial models, three initialization layers, 30 refinement steps per cycle, five maximum evolution cycles, the full candidate pool, and no method-pool evolution. Early stopping is enabled at zero loss. Every evaluation exhaustively visits the 20 ordered pairs $(p, g) \in \{-2, -1, 0, 1, 2\}^2$ with $p \neq g$, resetting the external position and target for each episode. A graph receives at most four executions to reach the target. Loss is mean absolute final distance; a successful run reaches the target from all 20 initial states. Evaluation performs no random sampling and checks that it leaves the search random-number states unchanged.

Search progression and accounting. Initialization alone succeeds in 20, 4, and 0 runs, respectively. Subsequent refinement raises these totals to 29, 15, and 1. Mean final accuracies are 0.9667, 0.5033, and 0.3200, with mean losses 0.0667, 0.9900, and 1.0667. Nominal caps are shared; early stopping and the available replacements produce the different actual evaluation budgets in Table 6. "Distinct graph" means an ordered attributed-graph signature, with literal display names removed; it is an accounting unit rather than a functional equivalence class.

The V0 early-stop path exits before updating some refinement counters and the best-model snapshot. We therefore evaluate the saved final graph and record every evaluation callback. Reconstructing accepted replacements from these callbacks gives totals of 9, 13, and 19, compared with native logged counters of 0, 1, and 18. This bookkeeping correction changes neither the trained graphs nor their measured outcomes. The earlier ten-seed experiment in Appendix L.4 uses a different episode-

Table 7: Matched state encoding results using V0. Ratios include the common adapter interface and state-token construction/extraction.

Quantity	Result
Candidate/final-model records	1,007
Paired initial conditions	20,140
Paired environment steps	70,670
Native graph executions, both encodings	141,340
Numerical, state, trajectory, and metric comparisons	All matched
Mean additional state edges, final graphs	2.189
Median explicit/implicit elapsed-time ratio	1.137
Ratio interquartile range	1.086–1.197

sampling protocol and remains a separate experiment. The present result establishes improved search success from a role-sensitive type prior in this fully enumerated task.

C Matched Explicit and Implicit State Encodings

Explicit state threading and implicit external-state access reproduce the same program behavior when they preserve the same state-access order. We verified this correspondence across all exported records from the type experiment, including unsuccessful candidates.

Construction. For each run and condition, the candidate catalogue contains the first 20 distinct ordered graph signatures encountered, where available, plus the final graph even when already recorded. The resulting 1,007 records include repeated final graphs. Every state-reading or state-writing method is translated to consume and return an immutable (p, g) state token. Token edges follow the original serial state-access order, including reads. Each access adds one state input/output port and one state edge; a final edge returns the resulting state. No computation node is added. Numerical argument binding and output aggregation remain unchanged.

Execution comparison. Both encodings use the original native serial executor on the same 20 initial conditions and four-step episode limit. Checks compare per-node numerical outputs, state after each access, collapsed output, each environment transition, and the recorded final loss and accuracy. During explicit execution, the global implicit state is deliberately assigned unrelated values, which verifies that the token-based methods neither read nor modify it.

Timing. Each of the 90 final graphs receives one complete warm-up and five paired timing repetitions over all 20 episodes; the order of the two encodings is randomized within each pair. The median ratio is computed per model and then summarized across models. It measures this Python object-port implementation, including state construction, dispatch, and extraction. Both conditions use the same already-searched candidates, so the experiment isolates encoding and execution; independent search over explicit-state programs is a separate comparison.

D Controlled Single-Graph Execution Scaling

At fixed depth and batch size, the native executor processes graphs from 32 to 1,024 operations with consistent numerical results. Over this 32-fold increase in operation count, median serial time increases from 2.127 to 70.586 ms. Type compatibility and topological preparation remain small relative to graph execution throughout the measured range.

Graph and workload design. Graphs have eight layers of width $w \in \{4, 8, 16, 32, 64, 128\}$, four input features, and batch size 32. There are $8w$ operation nodes, two boundary nodes, and $2(8w) + w$ edges. Each operation has two inputs. Its first predecessor slot uses a permutation of the preceding layer, ensuring every earlier node contributes to an output, while the second slot selects a distinct predecessor. The first layer selects two of the four input features. Final-layer outputs are aggregated with the native average operation.

Table 8: V0 execution scaling, in milliseconds for batch size 32. Values are medians over five graph-level timing medians. Speedup is the median of paired serial/threaded ratios, so it need not equal the ratio of the displayed medians.

Operations	Edges	Type pass	Topology	Serial	4 threads	Speedup
32	68	0.051	0.090	2.127	3.478	0.610
64	136	0.096	0.164	4.086	5.782	0.697
128	272	0.181	0.305	8.022	10.482	0.754
256	544	0.367	0.598	16.894	20.436	0.805
512	1,088	0.753	1.163	32.621	40.360	0.808
1,024	2,176	1.448	2.310	70.586	82.379	0.811

Each scalar operation starts from $z = 0.6a + 0.4b$ and performs 16 iterations of $z \leftarrow 0.98z + 0.02\tanh(0.9z + 0.05)$. This is ordinary CPU arithmetic, with vectorization disabled. Five independently wired graphs are evaluated at each width. Two disjoint pilot graphs precede the frozen formal protocol. The input seed is 2026110600, and formal graph seeds at width w are $51000 + 1000w + i$, $i \in \{1, \ldots, 5\}$.

Measurement and validation. Each metric receives two warm-ups and seven timed calls, with shuffled case and metric order. Timing uses a monotonic high-resolution clock without memory tracing. The environment is Windows, Python 3.12.9, NumPy 2.1.3, and NetworkX 3.4.2, on a 16-core/24-thread Intel host. Serial timing includes topological sorting; threaded timing includes dependency bookkeeping and thread-pool creation/destruction. The separate topology column is not deducted from either. The type pass is an additional per-edge measurement calling the native type compatibility predicate on declared ports, including the final aggregation contract. It is distinct from an integrated whole-program compiler check.

All 30 graphs satisfy the specified counts, depth, acyclicity, input slots, and declared type compatibility. A deliberately incompatible signature is rejected. Both execution paths agree with an independently implemented NumPy reference at tolerance 10^{-12}; maximum observed reference error is 2.22×10^{-16}. Separately traced allocation peaks exclude pre-existing graphs and inputs, and measure Python allocations rather than total resident memory. At 1,024 operations, the median serial and threaded peaks are 3.60 and 5.33 MB, respectively.

Observed scaling regime. Four Python threads are slower than serial execution for this scalar CPU workload. Together with the earlier simulated I/O experiment, the measurements show how operator cost and GIL behavior determine whether available graph parallelism improves latency. The largest tested graph has 1,024 operations; the point at which preparation dominates has not been reached. Node and edge counts grow together at fixed arity, so this experiment measures a controlled width expansion rather than an independent edge-density sweep.

E Shared-Feature Graphs for Action Selection

A shared-feature graph provides a concrete connection between numerical computation, persistent state, and discrete actions. One feature can contribute to several action scores while a mutually exclusive routing stage selects a zero-output action node. The same organization supports the four evaluated task families.

Controller representation and optimization. For observation o_t and previous internal state h_t, the graph computes

$$z_t = \phi_G(o_t, h_t), \qquad q_{t,a} = \sum_j w_{a,j} z_{t,j}, \qquad a_t = \mathrm{route}(q_t, u_t), \tag{8}$$

where u_t supplies any stochastic routing draw. Feature calculations read the old state, and a single commit stage writes the state used on the next step. The shared configuration has four storage slots arranged as two independent state channels. A decay factor of 0.9 and clipping are part of its specified update rule.

Table 9: Held-out completion rates (%) of the selected controller portfolio. Each cell uses 2,048 episodes; all five training seeds are shown. The mean row averages training seeds within each task.

Training seed	Catch	Maze	Snake	Bird
101	99.22	96.09	91.65	100.00
102	61.28	75.98	0.78	100.00
103	100.00	98.88	32.18	100.00
104	100.00	96.24	93.12	100.00
105	100.00	96.53	0.05	100.00
Mean	92.10	92.74	43.55	100.00

The training procedure provides the shared-feature interface, per-action score branches, route selection, and state-update rule. CMA optimizes the score coefficients. Configurations permitting growth propose eight additional arithmetic features at each of the prescribed 15%, 35%, 55%, and 75% budget milestones and retarget state-emission sources. Thus learned coefficients and proposed features sit within a declared controller template. Models are selected by validation success, with progress breaking ties. Each task uses its own selected model; the task identity is supplied externally.

The bird results are consistent across seeds, while Snake shows substantial variation. In the validation-selected representative system (seed 104), Catch, Maze, and Snake have no subsequent feature-growth event; Bird has four. Its controller contains 44 features and 88 coefficients. One saved feature is $z_{16} = \mathrm{gapBottom} + h_{1,t}$ and contributes with coefficients about $+3.001$ and -6.538 to the flap and coast scores. This is a direct example of shared numerical computation feeding distinct action branches.

Execution on the original ATF backend. The frozen representative Bird graph has 314 nodes and 411 edges. An explicit adapter expands the input, constant-trigger, state-commit, and observation interfaces, giving 316 nodes and 565 edges. Every feature operation, score coefficient, product, and accumulation is represented by graph operations. The commit stage sends a completion token to the router; enabled zero-output action sinks append action receipts to the step context, and the environment advances after graph execution.

We replay the first 20 consecutive episodes of the pre-existing test stream (environment seed 200000104; episode indices starting at 83000000), using the same external random draw in all executors. Across 20,480 steps, V0 serial execution agrees exactly in features, scores, state, actions, and trajectories with the portable interpreter and the original kernel function body. A further 2,048 of these steps match the previously archived trace. All 49 boundary probes and their 49 insertion-order-reordered counterparts pass. This replay establishes executable transfer of the frozen controller through the stated adapter; the performance results in Table 9 come from the preceding training and held-out evaluation.

F Sensitivity to the Atomic Method Pool

Expanding the atomic pool changes search quality non-monotonically. The best aggregate numerical result in this comparison comes from the six-operation core, while the best maze result comes from the nine-operation extension. The comparison covers nine pools spanning 6–20 operations.

Pools and search protocol. The core contains identity, addition, subtraction, multiplication, safe division, and duplication. The nested extensions add increment, decrement, and negation (9 operations); square, ReLU, absolute value, three-input mean, maximum, and minimum (15); then sine, cosine, two-argument arctangent, scaling, and safe square root (20). Five further pools vary polynomial, reciprocal, and trigonometric combinations. All use the same path-feedback procedure: each distinct executed route receives one terminal-loss update per episode, with exponential averaging coefficient 0.08, temperature 0.12, and exploration fraction 0.2. Candidates grow from identity scaffolds with periodic extension.

Numerical tasks use 131,072 training input presentations per task and seed, sampled from 2,048 fixed training examples, 256 candidate opportunities, and a 32-node limit per graph. Twelve tasks are

Table 10: Atomic-pool sensitivity. Numerical values average ten task families and four seeds. Maze values average two seeds under within-episode route switching. Both columns are percentages with their own evaluation units.

Pool	Operations	Numerical input pass rate	Maze completion
Arithmetic	6	14.01	11.43
Square	7	11.47	17.97
Basic	9	11.45	58.01
Nonlinear	15	7.72	37.89
Extended	20	11.35	6.93
Algebra	8	11.32	12.89
Polynomial	9	8.57	31.84
Balanced	12	13.62	27.44
General	16	10.83	14.16

grouped into ten equally weighted families, averaging the three equivalent two-variable multiplication tasks before aggregation. Four training seeds are used. Testing uses 1,024 inputs; the reported pass rate is the fraction with relative error at most 1%. Action tasks use 65,536 training episodes per task and seed, 128 candidate opportunities, a 24-node graph limit, and two seeds. Checkpoint evaluation uses 512 episodes per route-execution mode. Candidate generation, pool composition, and path feedback define this subsequent search host.

The numerical comparison comprises 432 records, including 32 reused baseline records; the action comparison comprises 144 newly trained runs. The broader 928-run study additionally includes mechanism ablations, so that total is not the number of pool-size comparisons. More operations offer new expressions while increasing the alternatives explored within the same nominal budget. Because operator identities and arities change with the extensions, the table describes practical pool sensitivity. Numerical input pass rates and action completion rates are reported separately, preserving their distinct meanings.

G Large Continual-Learning Execution Workload

The ATF execution core also supports a sustained workload with 80 complete learning sequences and 3,200 stages. Across training and checkpoint evaluation, 239,206,400 input rows are checked against native ATF outputs. This measures the substrate's use inside a long-running application, complementing the single-graph measurements in Appendix D.

Environment and design. Each observation has 64 fields, one of 32 identities, and six available actions. The 40-stage sequence first presents 20 deterministic rule families and then returns to them in reverse order. Fields and identities are assigned by seed. Training uses 3,072 interactions per stage, at most 2,048 candidates per search refresh, 6,144 stored observations, and a 48-derived-node cap per candidate graph. Ten seeds are used: 41113, 41213, 41317, 41423, 41527, 41631, 41741, 41847, 41957, and 42061.

Six mechanisms are compared at concept-library capacity 32: direct reuse, small-set retrieval, retrieval with prediction history, reserved fresh search, reward/complexity selection, and their full combination. Direct reuse and the full combination are additionally run at capacity 128, yielding 80 sequences. Paired conditions share observations, exploration draws, action sets, and nominal training and candidate limits. ATF executes the risk-expression graph, identity lookup, and action-value computation. Candidate proposals, selection, and library management are supplied by the surrounding learner.

Counting execution. The completed runs contain 9,830,400 effective training actions and 229,376,000 checkpoint-evaluation rows. Their sum is the 239,206,400 native-checked input rows reported above. One graph call can receive a NumPy array containing many rows, and the action-value head can perform a matrix computation within one node. Consequently, input rows, graph calls, concept entries, and graph nodes are distinct scale measures. The independent training units remain the ten seeds.

Table 11: Continual-learning results across ten paired seeds. Lower cost and training loss are better. Loss is accumulated reward shortfall relative to an oracle action over the 40 stages.

Mechanism (capacity)	Mean cost	Return cost	Training loss	Mastered/40
Direct reuse (32)	1,792.32	1,749.76	62,809.12	21
Small-set retrieval (32)	1,777.60	1,762.56	62,942.44	21
Retrieval + history (32)	1,720.32	1,648.00	62,617.83	23
Reserved fresh search (32)	1,817.92	1,739.52	63,083.45	21
Reward/complexity (32)	1,800.64	1,767.68	62,681.52	21
Full combination (32)	1,855.04	1,802.88	63,203.22	20
Direct reuse (128)	1,691.52	1,512.96	61,926.77	21
Full combination (128)	1,683.20	1,472.00	61,921.29	21

Learning outcomes. Learning cost is the earliest checkpoint after which both balanced risk accuracy of at least 90% and optimal-action rate of at least 80% in the carrying condition are maintained. Unsuccessful stages receive the fixed cost 3,200. A rule/first-or-return configuration counts as mastered when at least eight of ten seeds meet the criterion, giving 40 configurations in total.

The prespecified advantage criterion for the full combination at capacity 32 requires at least eight of ten seeds to reduce cost by at least 10%, without increasing average training loss. It is not met: zero seeds attain the cost threshold, mean cost increases by 3.50%, and loss increases by approximately 0.63%. Retrieval with history has the lowest mean cost among the capacity-32 conditions. Higher-order parity remains difficult: the three- and five-input XOR rules have approximately chance-level risk accuracy, with no seed reaching the mastery criterion on first or return exposure.

Completion audit. An initial 16 MiB serialized-state guard stopped part of the experiment. The formal completion uniformly raises that guard to 256 MiB while retaining the training, candidate, and concept budgets. Fifty-two completed sequences are preserved with identical file hashes; the remaining 28 start from their initial conditions. Three discarded prefixes account for 343,296 additional actions, recorded separately from the effective training count. Final frozen replay passes for all 80 models. Peak serialized state for the full combination is approximately 4.67 MiB at capacity 32 and 18.67 MiB at capacity 128.

H Reusable-Library Capacity and Return to Earlier Tasks

Larger libraries help most when earlier tasks must be recovered after a longer sequence. In the 40-stage experiment, increasing capacity from 32 to 128 lowers mean cost by 5.62% for direct reuse and 9.26% for the full combination. For the full combination, return-stage cost drops from 1,802.88 to 1,472.00 (18.35%), whereas first-exposure cost changes from 1,907.20 to 1,894.40. At the same capacity 128, the full combination and direct reuse are close: their mean cost difference is -8.32, with an exploratory paired-bootstrap 95% interval of $[-103.68, 87.68]$. The larger-storage result therefore concerns capacity and reuse over time.

Paired short-sequence comparison. A complementary experiment changes only the library capacity to 8, 16, 32, or 64, retaining the learner, tasks, admission and replacement rules, search budgets, and five training seeds. Three five-stage relationship sequences per seed give 60 sequences, 300 stages, and 460,800 interactions. Transfer cost averages the new-identity and return stages; a failure receives the fixed cost 1,600. This is a paired capacity sensitivity test on the same seeds.

Increasing capacity in these short sequences produces no stable learning gain. All three larger-capacity conditions have zero of five seeds achieving the prespecified 20% transfer-cost improvement. Capacities 32 and 64 generate identical models, learning curves, libraries, and action trajectories in all 15 paired sequences because occupancy never exceeds 21 entries. Meanwhile, the mean final single-model derived-node count is 15.87, 15.20, 15.19, and 15.19. Library capacity controls retained reusable structures; the final selected graph can remain small even when the collection of stored structures grows.

Table 12: Short-sequence library capacity comparison. The actual peak counts show whether the configured limit was reached. All capacities master 14 of the 15 sequence-stage configurations under the frozen criterion.

Capacity	Transfer cost	Training loss	Mastered/15	Peak entries
8	226.13	2,993.39	14	8
16	238.93	3,000.39	14	16
32	234.67	2,997.98	14	21
64	234.67	2,997.98	14	21

Evidence organization. The four V0 supplement records are organized under the relative artifact directories `experiments/rebuttal_20261106/type_search`, `state_encoding`, `scaling`, and `action_example`; each contains its protocol, source-version record, and measurements. The subsequent studies are identified by their controller portfolio, atomic-pool sensitivity, 40-stage continual-learning, and short-sequence capacity protocols. Their source snapshots and row-count audits distinguish training, evaluation, and diagnostic records. The results above use only completed formal records; planned extensions and withdrawn records do not enter the tables.

I Typed-Graph Comparisons

Matched search and library growth. We cross frozen ATF V0 versus an independent reference DAG backend with fixed versus growing method libraries. This reference backend is distinct from the PyTensor system below. Both backends use the same candidate proposer and module admission rule, eight feed-forward genome slots, and a cap of eight expanded active primitive operations. Module calls are charged for their internal operations. All targets have primitive-only witnesses within the common cap. Every port is declared `scalar`. The affine stages target $2(x+1)$, $2(x+1)+3$, and $x+2.5$, using 17 training and 32 disjoint verification points. Parity stages target the first two, three, and four bits of a four-bit input; the complete 16-row truth table is used, without a held-out generalization claim. After stages 1 and 2, a training-successful active graph containing at least two primitive operations is admitted as a module in both growing-library arms.

The primary protocol uses 30 shared seeds and 256 evaluations per stage, including evaluations after success. All 360 paired stages have identical candidate traces, selected graphs, libraries, and outputs. A separate parity budget check, specified after the primary results, uses 1,024 evaluations and the same seeds. Its 180 paired stages also agree. Independent numerical replay checks all 184,320 primary and 368,640 secondary evaluation records; these records and paired backends are not additional independent search seeds.

Table 13: Search successes out of 30 seeds. ATF and the reference DAG have the same counts in every row. The 1,024-evaluation rows are a separate post-primary budget check, not additional seeds.

Family	Stage	Evaluations/stage	Fixed library	Growing library
Affine	1	256	26	26
Affine	2	256	20	23
Affine	3	256	2	10
Parity	1	256	5	5
Parity	2–3	256	0	0
Parity	2	1,024	0	6
Parity	3	1,024	0	1

The affine improvement belongs to the common library-growth mechanism. Its mean training primitive-call count over all three stages increases from 36,007.7 to 40,847.6. In the primary study, median paired ATF/reference ratios for setup, compilation, training execution, and admission range from 3.05 to 5.56. These are implementation timings: the reference compiles modules into flat instruction sequences, whereas V0 uses nested `EvolvedMethod` interpretation. Complete costs, Wilson intervals, protocols, and traces are provided under `experiments/typed_dag_comparison_20261108/`.

Concrete system workflow and action interfaces. The PyTensor 3.3.8 comparison uses native `OpFromGraph` modules and ATF V0 `EvolvedMethod` modules. A common enumerator discovers each affine stage, registers successful graphs, and reuses them in later stages, with limits of four top-level calls, eight expanded primitive calls, and 512 candidates per stage. Twelve order-seeded cases cover eleven distinct primitive permutations; the duplicate is retained. All 36 paired stages and 2,638 candidate evaluations agree. Every second stage calls the first-stage module, and every third stage reuses the second stage and its nested first-stage module. All 24 disk snapshots are reconstructed through fresh backend instances. This persistence uses common JSON declarations, rather than either framework's native object serialization.

Frozen Catch, Maze, Snake, and Bird controllers are also expressed with native PyTensor numerical nodes and compared step by step with freshly executed ATF V0 and the reference kernel. Consecutive evaluation prefixes contain 320, 163, 237, and 20,480 steps, respectively. All 21,200 steps agree in actions, permits, and environment state; the maximum error in features, scores, and memory is zero. Bird additionally matches its archived trace, and all 49 routing/state boundary probes pass. These are execution checks of fixed learned models, with environment advancement outside both controller graphs, not retraining results. Hashes of 38 ATF and 405 PyTensor Python source files remain unchanged.

ATF executes connected zero-output action sinks. The tested PyTensor paths keep action calls reachable through requested outputs or shared-variable `updates`; the latter permits empty user-visible outputs. The public `wrap_py` decorator also wraps ordinary Python action callbacks. Both paths preserve actual environment changes in small action probes with the Python linker and either no optimizer or `FAST_RUN`; unanchored callback branches are not executed. Full controllers are checked only with optimization disabled. Retaining an action does not determine the order of conflicting actions or establish concurrency safety. Native interface probes also expose a V0 boundary: its scalar-output module wrapper rejects vector results that PyTensor's native module accepts. Protocols, adapters, and audits are provided under `experiments/typed_dag_workflow_20261108/`.

Original Experiments and Technical Details

The following sections retain the protocols, principal results, and semantic definitions needed to assess the original experiments. Extended derivations, operator listings, complete plotting material, and implementation notes are provided in `Docs/extended_materials/` in the anonymous repository. The controlled measurements and application studies in Appendices A–I retain their separate protocols and execution-version records.

J Related Work and Representation Interfaces

Typed program graphs and reusable intermediate values are established in strongly typed GP, CGP, and MT-CGP Montana (1995); Miller & Turner (2015); Harding et al. (2012). LLVM SelectionDAG separates typed data values from chain dependencies for side-effect ordering LLVM Project (n.d.). AdaptoFlux applies this separation to a symbolic-synthesis interface: data edges bind arguments of registered host-language callables, while the saved execution contract orders declared conflicting resource accesses. Candidate evaluation, graph modification, subgraph extraction, and persistence maintain that contract (Appendix A).

DreamCoder learns reusable abstractions in a functional program space Ellis et al. (2021); mutable interaction can also be expressed by passing explicit state values. Appendix C compares that interface with implicit accesses for the same programs and selected access order. Popper learns logical programs through a generate–test–constrain loop Cropper & Morel (2021); its failure-derived constraints differ from the current supplied type declarations and search metadata. These approaches provide alternative interfaces and search mechanisms, rather than a separation in computability.

PySR, gplearn, and AI Feynman specialize in recovering numerical expressions Cranmer (2023); Udrescu & Tegmark (2020). Symbolic policy search and language-model-guided discovery emphasize candidate generation and optimization Landajuela et al. (2021); Romera-Paredes et al. (2024). AdaptoFlux's common registration and execution interface accommodates several such choices;

the current demonstrations use LayerGrow, GraphEvo, and a CMA-based controller optimizer. Appendix N retains the restricted static-regression comparison, including its negative results.

K Execution and Search Cost

K.1 Inference Cost

For V operation nodes, E routed data edges, N samples, and average operation and routing costs C_{op} and C_{route}, the original backend separates input conversion, topological preparation, node execution, and output aggregation. Topological sorting costs $O(V + E)$; executing a fixed scalar graph contributes $O\big(N(VC_{\mathrm{op}} + EC_{\mathrm{route}})\big)$. Input conversion and output aggregation additionally depend on input/output size and the selected aggregation rule. These terms describe the original backend used in the historical and Appendix D measurements.

The revised runtime prepares the saved semantic order and resource-control dependencies before execution. Its scheduler operates on data and control dependencies together. For stateful batches with effect ordering enabled, it completes one sample's graph before the next; pure graphs use the batched traversal. Section 3.4 states this revised cost decomposition. The original timings do not measure the new planning pass.

K.2 Navigation Procedures

Search cost is the sum of candidate construction or modification, evaluation, and acceptance/bookkeeping costs over the candidates actually examined. LayerGrow repeatedly extends and evaluates a graph, with rollback after unsuccessful proposals. Its growth-attempt cap therefore need not imply a reached graph depth. GraphEvo evaluates replacements on a fixed skeleton and can extract reusable subgraphs. CombinedTrainer alternates growth and refinement with optional pool filtering. Candidate counts, graph sizes, episode lengths, and stopping rules jointly determine their realized costs; the protocols below retain these quantities separately from the type checks and execution costs.

K.3 Backend Constraints

Available concurrency is bounded by dependency width and critical-path length. Thread-pool creation, ready-node management, Python dispatch, and intermediate storage add backend costs. Simulated waiting operations and Python scalar CPU operations exhibit different speedups (Appendices L.3 and D). Type compatibility checks do not detect shared-state conflicts; those are governed by the separate execution contract. Stage-by-stage cost derivations and profiling figures are retained in `Docs/extended_materials/`.

L Original Experimental Protocols and Results

These experiments use the original backend. The following tables report their original evaluation units and stopping criteria; they are not pooled with the later controlled studies in Appendices B–H.

L.1 LayerGrow on Titanic

The historical runner loads preprocessed Titanic observations from `train_processed.csv`; the reported accuracy is the search-time score on the observations supplied to the trainer. This is a structural-exploration probe rather than an independent generalization evaluation. The pool has nine scalar operations: identity, two-output copying, addition, subtraction, multiplication, protected division, increment, decrement, and negation. Configured maximum depths are $5, 10, \ldots, 50$, with ten runs per setting, ten attempts per layer, rollback by two layers, decision threshold zero, and a total cap of 2,500 growth attempts. The historical seed expression is $42r + \mathrm{hash}(\texttt{max_layers_}L) \bmod 1000$ for $r = 1, \ldots, 10$; reproduction therefore also depends on Python's hash initialization.

At requested depths 45 and 50, all runs exhaust the budget before attaining those depths. This locates a search-budget bottleneck rather than failure to execute a graph of that size. The source is `experiments/LayerGrowTrainer_exp1/LayerGrowTrainer_exp1.py`; `main.py`

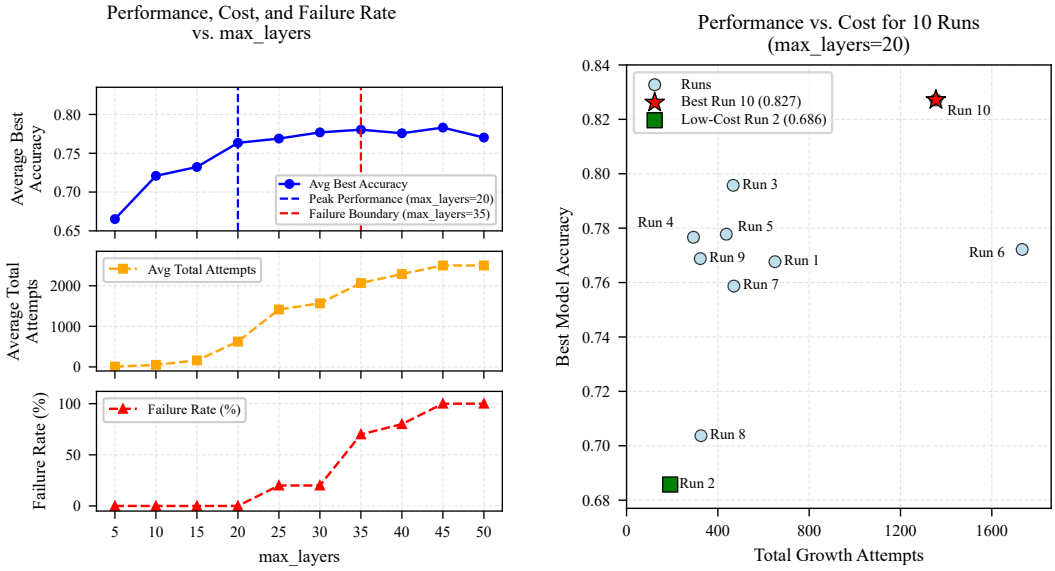


Figure 4: Original LayerGrow study: configured-depth sensitivity and variation among ten runs at a maximum of twenty layers. The configured maximum and achieved depth are distinct.

Table 14: LayerGrow results over ten runs per setting. Accuracy is the best observed value per run, including runs stopped by the attempt cap. Exhaustion means that all 2,500 attempts were consumed.

Max. depth	Accuracy (mean ± SD)	Reached depth	Attempts	Exhaustion (%)
5	0.665 ± 0.042	5.0	6.5	0
10	0.721 ± 0.030	10.0	50.5	0
15	0.732 ± 0.036	15.0	160.5	0
20	0.763 ± 0.041	20.0	624.5	0
25	0.769 ± 0.028	22.9	1414.9	20
30	0.777 ± 0.025	26.3	1567.4	20
35	0.780 ± 0.022	21.7	2069.2	70
40	0.776 ± 0.020	23.3	2289.7	80
45	0.783 ± 0.024	19.6	2500.0	100
50	0.770 ± 0.032	18.7	2500.0	100

in that directory aggregates the outputs. The configuration/run summary is `experiment_results_summary.csv` in the same directory. Large per-attempt logs are generated by the runner. Its historical runtime import requires the matching original API when reproducing the experiment.

L.2 GraphEvo and Reusable Subgraphs

The task chain is $f_1(x) = 2(x + 1)$, $f_2(x) = f_1(x) + 3$, and $f_3(x) = 0.5 f_2(x)$. All methods use scalar interfaces. The minimal pool contains increment, multiply by two, add three, multiply by one half, and identity; the extended pool adds `add_2`, and the static-injection condition adds the exact f_1 method to the minimal pool. There are 100 repetitions per configuration, with NumPy seeds $42 + \text{rep_id}$ and data resampled per repetition. Tasks 1/2 use maximum initial depths two or three; Task 3 always permits at most three top-level calls.

Sample accuracy uses relative error below 1%, with denominator $|y| + 10^{-8}$. Reported Task 3 success requires accuracy > 0.95; the stricter complete-chain criterion additionally requires Tasks 1/2 to reach accuracy ≥ 0.999. The two counts are distinct.

Exact enumeration shows that the fixed minimal and extended pools require four calls to express $x + 2.5$. Composite methods can make it reachable within Task 3's three-call bound. The comparison

Table 15: Original three-task reuse study, 100 runs per configuration. The reported success criterion concerns Task 3. Static injection supplies f_1 directly; evolution extracts methods during training.

Pool updating	Pool	Task 1/2 depth	Task 3 accuracy	Successes/100
Disabled	Minimal	2	0.0006 ± 0.0021	0
Disabled	Extended	2	0.0003 ± 0.0012	0
Disabled	Minimal	3	0.0007 ± 0.0022	0
Evolution	Minimal	2	0.1308 ± 0.3360	13
Evolution	Extended	2	0.1312 ± 0.3359	13
Evolution	Minimal	3	0.0908 ± 0.2859	9
Static injection	Minimal+f_1	2	0.1206 ± 0.3247	12

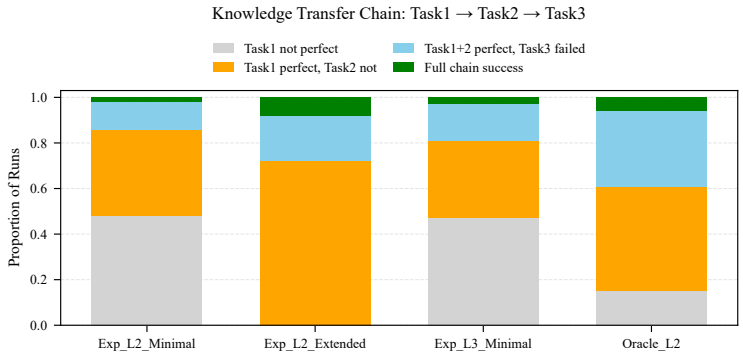


Figure 5: Complete and partial transfer paths in the original three-task experiment. Green denotes the stricter complete-chain criterion.

therefore measures abstraction at fixed top-level depth; Appendix I separately matches expanded capacity. Full-chain success remains limited. The implementation is `experiments/GraphEvo_exp3/exp3_main.py`; configuration statistics are in `exp3_results/summary_table.csv` below that directory. Complete model and pool histories are regenerated by the script.

L.3 Topology-Aware Task Parallelism

Thirty saved graphs from the LayerGrow study are evaluated using a dynamic `ThreadPoolExecutor` with $1, 2, 4, 8, 16$ workers. Their actual depths span 2–35; graph identities and topology statistics are retained in the extended material and the original benchmark records. The same input batch and method pool are used across worker counts, with seed 42. Two workload modes are distinguished: lightweight Python computation, and the same methods with a 1 ms `time.sleep` inside each method to simulate non-blocking waiting.

For the waiting workload, shallow wide graphs attain a mean $10.25\times$ speedup with 16 workers. Across all 30 graphs at 16 workers, speedup correlates with average width ($r \approx 0.99$), whereas correlations with node count and depth are approximately -0.16 and -0.41. These measurements characterize the simulated waiting workload. The separate CPU experiment in Appendix D reports slower threaded execution for Python scalar operations. Source configurations, the 30-graph inventory, structural correlations, and profiling plots are available under `experiments/PipelineParallel_exp2/` and `Docs/extended_materials/`.

L.4 Original Minimal Stateful Synthesis Experiment

Position and target are drawn from $\{-2, -1, 0, 1, 2\}$ with unequal initial values. The graph rereads observations after each state-changing execution, with at most four steps per episode. Each final graph is evaluated on 100 randomized episodes; a search run succeeds when at least 95% reach the target. Training probes use 20 episodes per evaluation. Both conditions use seeds 42–51 and the same nine method implementations; only declared types and the associated root-input signatures

Table 16: Original stateful experiment: final success percentages over 100 episodes for each search seed. A successful search requires at least 95%.

Seed	42	43	44	45	46	47	48	49	50	51
All scalar	33	30	22	42	24	37	32	27	24	32
Typed roles	100	100	0	100	0	100	100	100	100	100

Table 17: Original geometric-classification results: test accuracy (mean ± SD) across 100 repetitions. "Disabled", "once", and "periodic" refer to pool filtering in the three-cycle CombinedTrainer.

Configuration	Circles	Spirals: full	Spirals: restricted
Combined: disabled	0.8909 ± 0.0837	0.8493 ± 0.0338	0.8386 ± 0.0390
Growth then refinement	0.7927 ± 0.1354	0.8148 ± 0.0556	0.7989 ± 0.0421
Growth only	0.7725 ± 0.0622	0.8053 ± 0.0389	0.8019 ± 0.0355
Combined: once	0.7472 ± 0.1547	0.7823 ± 0.0793	0.7791 ± 0.0644
Refinement only	0.6985 ± 0.1080	0.7624 ± 0.0798	0.7326 ± 0.0992
Combined: periodic	0.6292 ± 0.1424	0.7209 ± 0.1094	0.7381 ± 0.0762

differ. The pool contains state readers, direction calculation, the goal-aware movement operation, and distractors.

GraphEvo uses fixed three-layer initialization, three initial skeletons, up to 30 replacements per cycle and five cycles, the full candidate pool, disabled method-pool evolution, and `early_stop_eps=0`. The reported graph is the final graph of each search. These randomized ten-seed results are separate from the exhaustive thirty-seed protocol in Appendix B.

Typed roles succeed in 8/10 searches versus 0/10 for all-scalar declarations. Mean final accuracies are 0.800 ± 0.400 and 0.303 ± 0.060, respectively. The two typed failures omit the movement operation and never change state; the scalar graphs contain movement but omit the intended direction calculation. These observations isolate the supplied type-routing prior in this task.

Paired test and records. The shared seeds form ten pairs, with eight successes unique to the typed arm, zero unique to the scalar arm, and two shared failures. The exact paired McNemar test gives one-sided $p = 2^{-8} = 0.00390625$ and two-sided $p = 2^{-7} = 0.0078125$. The configurations and per-run graphs are under `experiments/minimal_stateful_comparison/`; `run_comparison.py` reproduces the comparison. The original source log is `logs/experiment_log_20260920_124456.log` in that directory.

M Geometric Classification and Pool Composition

The original study compares six search configurations on concentric circles, two spirals with a full pool, and two spirals with a restricted pool. Each configuration has 100 repetitions, seeds incremented from 42, 800 training and 200 test examples, and standardized inputs. The restricted pool contains 15 scalar operations: the nine operations in Appendix L.1, plus square, ReLU, absolute value, mean of three values, maximum, and minimum. The full pool adds sine, cosine, `atan2`, protected square root, and scaling.

LayerGrow alone permits 15 layers; the other growth configurations permit 10, with a 250-attempt cap. GraphEvo alone permits ten rounds; the combined conditions use eight refinement rounds per cycle. CombinedTrainer runs three cycles. Pool filtering is disabled, applied once, or applied each cycle; its target size is eight, population 20, elite fraction 30%, mutation rate 15%, and data fraction 30%. Filtering uses four generations, reduced to two in the periodic configuration. The stopping tolerance is 10^{-6}.

The unfiltered combined configuration has the highest mean accuracy in all three settings, but reaches 99% test accuracy in only 24, 1, and 1 of the 100 runs. Thus the spiral tasks remain difficult despite the higher mean. Training budgets and filtering schedules differ between configurations; these results evaluate complete search configurations, not an isolated effect of

Table 18: Best-of-ten test MSE in the original static-regression experiment. Values retain the precision of the original summary tables.

Task	ATF first	ATF prod	ATF sum	gplearn	PySR
mc^2	2e+3	2e−11	8e+0	8.3e−12	8.3e−12
ma	3e−12	2e−13	4e+1	1.9e−13	2.1e−12
$0.5mv^2$	4e+3	2e−11	8e+2	3.4e−11	3.4e−11
q_1q_2/r^2	1e+1	9e+0	1e+1	6.8e−14	2.7e−13
nT/V	3e+4	2e+4	3e+4	1.0e−10	2.2e−10
IR	5e−13	2e−13	1e−12	2.2e−13	2.2e−13
$(v_2 - v_1)/t$	7e+0	5e+0	1e+1	3.9e−14	1.1e−13
VI	1e−12	8e−13	2e−12	8.0e−13	1.0e−12

Table 19: ATF numerical-match frequency across all ten runs. Numerical matching requires absolute error below 10^{-5} on every test sample. This is distinct from both MSE and the manual symbolic-equivalence check.

Aggregation	mc^2	ma	$0.5mv^2$	q_1q_2/r^2	nT/V	IR	$(v_2 - v_1)/t$	VI
first (%)	0	20	0	0	0	10	0	50
prod (%)	0	100	0	0	0	100	0	100
sum (%)	0	0	0	0	0	20	0	30

pool size. Detailed timings, selected graph plots, and decision-boundary figures are retained in `Docs/extended_materials/`; scripts are under `experiments/Combined_exp4_circles/`, `experiments/Combined_exp4_spiral/`, and `experiments/Combined_exp4_spiral_no_Trigonometric/`.

N Static Symbolic Regression with Restricted Method Pools

Eight formula-recovery tasks compare the original probes with gplearn and PySR Cranmer (2023), using arithmetic primitives `add`, `sub`, `mul`, and `div`. ATF's construction interface also supplies identity and copying operations. Output aggregation is separately varied among `first`, `prod`, and `sum`; arbitrary imported methods and stateful actions are not used. Each condition has ten independently seeded runs. The tasks are mc^2, ma, $0.5mv^2$, q_1q_2/r^2, nT/V, IR, $(v_2 - v_1)/t$, and VI.

The specialized symbolic solvers obtain much lower error on Coulomb, ideal-gas, and acceleration tasks. ATF's product aggregation consistently recovers the simple multiplication tasks, whereas the other modes have lower numerical-match frequencies. Best-of-ten MSE alone therefore does not characterize robustness. For the kinetic-energy task, the constant 0.5 is supplied as an input feature. Manual inspection classifies the best ATF product expressions as algebraically equivalent for mc^2, ma, $0.5mv^2$, IR, and VI, but not for the other three tasks. Numerical matching and symbolic equivalence use different criteria; neither is substituted for the other.

The full expressions, run-level distributions, timing tables, XGBoost comparison, and manual equivalence table are in `Docs/extended_materials/`. Executable baseline scripts and result records are under `experiments/ExternalBaselines/`. These restricted experiments evaluate particular optimizers and aggregation rules; they do not establish a search advantage of implicit state or of the representation over other typed graphs.

O Hand-Crafted Snake Controller

The original untrained Snake controller reads observations, computes hazards and food direction, and selects among `turn_left`, `turn_right`, and `go_straight`. Mutually exclusive Boolean permits ensure that one action updates the environment per tick. Numerical computation and action emission share the stored graph; the graph is executed again on the next observation. This illustrates

Table 20: Repository entry points. Paths are relative to the anonymous repository; each listed experiment retains its own configuration and output conventions.

Study/material	Entry point or directory
LayerGrow depth	`experiments/LayerGrowTrainer_exp1/`
GraphEvo reuse	`experiments/GraphEvo_exp3/`
Original parallelism	`experiments/PipelineParallel_exp2/`
Original stateful task	`experiments/minimal_stateful_comparison/`
Static regression	`experiments/ExternalBaselines/`
Four frozen V0 studies	`experiments/rebuttal_20261106/`
Extended technical material	`Docs/extended_materials/`

the execution interface, independently of the learned controllers in Appendix E. The graph, demo, and larger diagram are provided in `experiments/embodied_snake/` and the extended material.

P Implementation and Reproduction Map

The `@method_profile` decorator binds an implementation to positional input/output signatures and metadata. Data ports determine argument routing; `pure`, `reads`, and `writes` describe effects in the revised runtime. The complete source interfaces are the executable reference, under `ATF/methods/` and `ATF/GraphProcessor/`; `Docs/effect_execution_zh.txt` documents the optional effect-ordering mode.

The four V0 studies supply a frozen runtime snapshot, source/protocol hashes, raw records, an integrity verifier, and a launcher that writes reproduction outputs into a new external workspace. The package README distinguishes evidence auditing, execution replay, and fresh measurements. This preserves the historical measurements when the repository runtime changes. The V1 regression suite and frozen-model replay are reported separately in Appendix A. Large historical model dumps and per-attempt logs are regenerated by their scripts; available summary records and selected artifacts remain in the corresponding experiment directories.

Q Method Metadata and Runtime Semantics

Q.1 Functional Tags

Metadata may label a method as a numerical mapping, an action, or both. Such descriptive tags support application-specific pool views. A method that returns a value can still change state; a zero-output method can still perform an action.

Q.2 Reversibility Tags

Tags describing invertibility or reversibility are optional search annotations. They do not prove that a callable is invertible, determine its effect set, or replace the runtime's execution contract. Extended illustrative taxonomies are retained in the repository material.

Q.3 Task-Specific Pool Views

A filter ϕ selects $\mathrm{Q}_{\mathrm{view}}(\phi) = \{m \in \mathrm{Q} \mid \phi(\mathrm{M}_m)\}$. The view changes the operators exposed to search, while sampling weights change proposal probabilities within that view. Pool composition and search budgets must therefore be recorded together when comparing outcomes.

Q.4 Hypothesis Class and the Formal Role of Metadata

Equation 1 defines admissible data DAGs through port compatibility and acyclicity. Filtering restricts that class to $\Omega_{\mathrm{Q}_{\mathrm{view}},\Sigma} \subseteq \Omega_{\mathrm{Q},\Sigma}$. Stateful execution additionally depends on $P = (G, \pi, E_c)$: the frozen order and control dependencies derived from declared resource accesses. Changing an effect

Table 21: Distinct roles of interface checks, effect declarations, and search metadata.

Mechanism	Role
Port types and arity	Admit compatible data connections to argument positions.
Data-graph acyclicity	Admit a topological schedule for data dependencies.
Read/write resources and saved order	Preserve the chosen order of conflicting accesses when effect ordering is enabled.
Search tags and weights	Select and sample the methods considered by the optimizer.
Reuse metadata	Guide extraction and reuse; composite methods retain their effect contracts.

declaration can change the required schedule and hence the execution contract; changing a search weight only changes proposal preference.

Type-prior measurements appear in Appendix B; effect validation appears in Appendix A. The pool-composition studies evaluate complete search configurations rather than isolating only the metadata filter.

Q.5 Root and Output Interfaces

The root distributes the externally supplied input into declared typed ports. The output interface applies a separately configured aggregation rule to its incoming values, such as first output, sum, product, or identity. Aggregation is configured through the collapse manager rather than selected from the internal Method Pool. Its choice affects the represented input/output mapping and is reported explicitly in Appendix N. State changes can occur at action nodes even when those nodes do not lie on a path to an aggregated numerical output.

Q.6 Static Acyclicity and Runtime Loop Semantics

Let $P = (G, \pi, E_c)$ be a fixed successful per-step execution contract. Let the state s_t contain all persistent resources, and fix the input and random choices for each step. One evaluation defines Φ_P, and the next-observation interface defines Π:

$$s_{t+1} = \Phi_P(s_t, x_t), \qquad x_{t+1} = \Pi(s_{t+1}). \tag{9}$$

The numerical graph output need not encode the entire environment state.

Finite-unrolling proposition. For finite horizon T, this trace has an equivalent acyclic unrolling using T copies of $G^+ = (V, E_d \cup E_c)$, with complete state and observations carried between successive copies. *Proof.* Within each copy, all dependencies point forward in π; between copies, they point from step t to $t+1$. Lexicographic order by $(t, \text{position in } \pi)$ therefore orders every dependency forward. Induction on t establishes execution equivalence: identical initial state and inputs yield the same first transition, and each subsequent copy receives exactly the preceding state and observation. □

The interaction loop is supplied outside the stored graph. A registered callable may also invoke another graph or recurse internally; its outer effect declaration must summarize the resources it accesses. Enclosing-graph type checks do not prove termination or analyze arbitrary Python control flow.

R Challenges in Automated Synthesis of Stateful Policies

Early LayerGrow/GraphEvo control trials exhibited proxy-reward failures: Snake policies repeatedly turned to survive without collecting food, while Bird policies repeatedly jumped or failed to avoid obstacles. These exploratory observations concern the tested pools, rewards, and search configurations; they are separate from the formal minimal-stateful study and the later CMA controller results. The public Snake directory contains the fixed demo, not those training runs. The early Bird training code in `experiments/embodied_bird/` is an implementation reference whose API may require adaptation to newer runtime versions. Further qualitative notes are retained in `Docs/extended_materials/`.

S Prospective Integration of Language-Model Candidate Generators

A language model could propose graphs or graph edits from the task, registered method interfaces, and graph schema. AdaptoFlux could validate method names, arity, port types, and acyclicity, construct effect-control dependencies, and execute the proposal to return task scores and trajectories. Executable-candidate rate, task success, search budget, and generation/validation costs are suitable evaluation measures. This propose–validate–execute–feedback integration is a research direction, not a completed experiment.

T Limitations

The original probes are sensitive to initialization, operator coverage, aggregation, and credit assignment. Their results do not establish superiority over other typed graph synthesis systems. Effect guarantees require complete resource declarations and deterministic successful execution; static interface checks do not verify arbitrary Python effects. The largest tested graph is not a practical upper bound. The main-text limitations distinguish these issues from the separate questions of explicit-state search and runtime preparation cost.

U Broader Impacts

Inspectable program graphs can expose computations and action interfaces for review, but interpretability does not itself ensure correct or safe behavior. Registered methods may inherit data or design biases and may perform harmful external actions. Applications should validate their Method Pool, isolate execution where appropriate, and check action permissions and environment constraints before deployment. The experiments use public numerical data and synthetic environments, with no human-subject intervention.

V Discussion and Future Work

Output interfaces. The static-regression study shows that aggregation is part of the experimental configuration. Future search procedures can optimize explicit output-producing subgraphs while using identity aggregation, rather than assuming a fixed reducer.

Execution backends. Compiled or hybrid backends may reduce interpreter, scheduling, and storage costs. Their benefit must be measured against operator cost, graph concurrency, and memory traffic; a change of implementation language alone does not establish a speedup.

Reusable executable knowledge. The controller and library studies provide concrete settings for combining registered primitives with learned structure. Future work described in Section 5 is to retain subgraphs with applicability conditions and evidence, and revise them through interaction within bounded resources. Broader transfer and continually learning behavior require their own comparative evaluation.